

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

-----oo0oo-----



Báo Cáo Đồ Án Giữa Kỳ

Môn An Toàn Và Phục Hồi Dữ Liệu - CQ2022/22

Năm học 2025 - 2026

Giảng Viên: Đặng Trần Minh Hậu

Nguyễn Văn Quang Huy

Ngô Đình Hy

Nhóm Thực Hiện: 30052004

Mục Lục

Thành viên và phân chia công việc.....	3
Tổng quan kỹ thuật chi tiết về hệ thống tập tin NTFS.....	3
I. Cấu trúc Volume và Bảng MFT.....	3
MFT Zone (Vùng An toàn cho MFT).....	4
II. Thành phần Cốt lõi: Master File Table (MFT).....	4
Khái niệm then chốt: Thuộc tính (Attributes).....	4
III. Dữ liệu Nội trú (Resident) và Ngoại trú (Non-Resident).....	5
Dữ liệu Nội trú (Resident Data).....	5
Dữ liệu Ngoại trú (Non-Resident Data).....	5
IV. Các Metafile Quan trọng.....	6
V. Tổ chức Thư mục:.....	6
VI. Các tính năng cao cấp khác.....	7
VII. Tình Huống và biện pháp cụ thể.....	7
1. Mô tả sai về phân vùng (Partition).....	7
2. Tham số sai của Volume.....	8
3. Bảng thư mục và bảng Cluster sai.....	9
4. File/thư mục đã xóa.....	9
VIII. Demo gây hư hại và phục hồi.....	10
1. Mô tả sai về phân vùng (Partition).....	10
2. Tham số sai của Volume.....	14
3. Bảng thư mục và bảng Cluster sai.....	17
4. File/thư mục đã xóa.....	23
Tài liệu tham khảo.....	26

Thành viên và phân chia công việc

MSSV	Họ Và Tên	Mục Đóng Góp	Mức độ hoàn thành
22120136	Mai Nhựt Huy	Làm slide thuyết trình, người thuyết trình chính, demo Mô tả sai về phân vùng, Bảng thư mục và clutser sai.	100%
22120141	Võ Nguyễn Song Huy	Demo File/thư mục đã xóa.	100%
22120142	Vy Quốc Huy	Tìm hiểu nội dung, demo tham số sai của Volume chính sửa slide thuyết trình.	100%

Tổng quan kỹ thuật chi tiết về hệ thống tập tin NTFS

NTFS (New Technology File System) là hệ thống tập tin 64-bit hiện đại, mang tính "giao dịch" (transactional) và có khả năng tự phục hồi, được Microsoft giới thiệu cùng với Windows NT. Nó được thiết kế để thay thế hoàn toàn hệ thống FAT (File Allocation Table) cũ kỹ, giải quyết triệt để các vấn đề về bảo mật, giới hạn kích thước, và độ tin cậy.

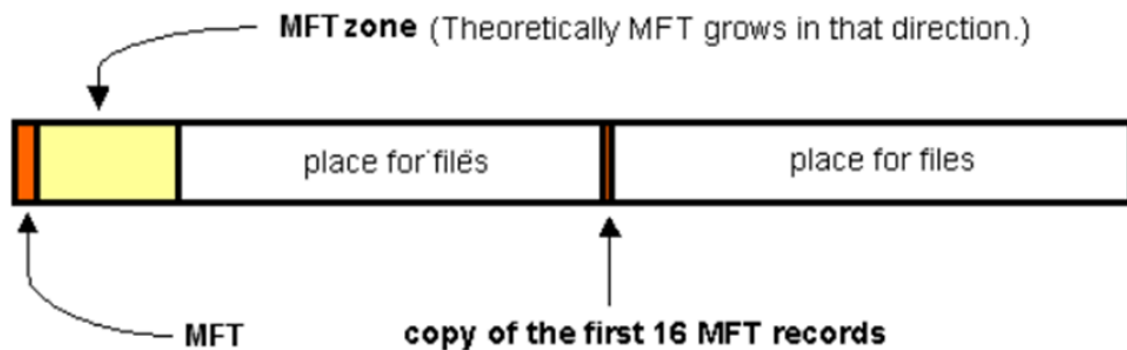
Nguyên tắc thiết kế cơ bản của NTFS là: **"Mọi thứ trên volume đều là một tập tin, và mọi tập tin là một tập hợp các thuộc tính (attributes)."**

Ngay cả các cấu trúc siêu dữ liệu (metadata) quản lý chính hệ thống tập tin—như bảng MFT, bản đồ cluster, nhật ký giao dịch—cũng được triển khai dưới dạng các tập tin đặc biệt (metafile).

I. Cấu trúc Volume và Bảng MFT

Khi một volume được định dạng NTFS, nó sẽ tạo ra một số cấu trúc cơ bản:

- **Boot Sector (\$Boot):** Chứa mã khởi động và các thông tin quan trọng (gọi là BPB - BIOS Parameter Block) mô tả volume, như kích thước cluster và quan trọng nhất là địa chỉ (cluster) bắt đầu của Master File Table (MFT).
- **Master File Table (MFT):** Đây là "trái tim", "bộ não" và "cuốn sổ cái" của NTFS. Nó là một tập tin đặc biệt (tên là \$MFT) đóng vai trò như một cơ sở dữ liệu quan hệ, chứa ít nhất một bản ghi (record) cho *mọi* tập tin và thư mục trên volume.
- **Data Area (Vùng Dữ liệu):** Phần còn lại của volume.



MFT Zone (Vùng An toàn cho MFT)

Để ngăn MFT bị phân mảnh (fragmentation), NTFS "dành" một vùng không gian ở đầu volume (thường là 12.5%) gọi là **MFT Zone**. Các tập tin thông thường không được phép ghi vào vùng này trừ khi toàn bộ phần còn lại của đĩa (87.5%) đã bị đầy. Điều này cho phép tập tin \$MFT (cơ sở dữ liệu trung tâm) phát triển một cách liên tục, giúp tối ưu hiệu suất truy xuất metadata.

II. Thành phần Cốt lõi: Master File Table (MFT)

MFT là một bảng không lồ, được chia thành các **MFT Record (Bản ghi MFT)**. Mỗi record có kích thước cố định, thường là **1KB** (1024 bytes).

- Mỗi khi bạn tạo một tập tin (hoặc thư mục), hệ thống sẽ cấp phát một MFT record mới cho nó.
- Bản thân MFT record **không chứa dữ liệu** theo cách hiểu thông thường. Thay vào đó, nó là một "vỏ chứa" cho một tập hợp các **Thuộc tính (Attributes)**.

Khái niệm then chốt: Thuộc tính (Attributes)

Mọi thông tin về một tập tin đều được lưu dưới dạng một thuộc tính. MFT record chỉ đơn giản là một danh sách các thuộc tính này. Các thuộc tính quan trọng bao gồm:

- **\$STANDARD_INFORMATION**: Thuộc tính bắt buộc. Chứa thông tin cơ bản như:
 - Ngày giờ (Timestamps): Bao gồm Created (Tạo), Modified (Sửa đổi nội dung), Accessed (Truy cập), và MFT Entry Modified (Sửa đổi metadata).
 - Thuộc tính DOS: Read-Only, Hidden, System, Archive.
 - Security ID tạm thời (sẽ được thay thế bằng con trỏ đến \$SECURITY_DESCRIPTOR).

- **\$FILE_NAME:** Tên của tập tin. Một tập tin có thể có **nhiều** thuộc tính \$FILE_NAME, ví dụ:
 - o Một tên dài (ví dụ: BaoCaoTaiChinh.docx).
 - o Một tên ngắn 8.3 (ví dụ: BAOCAO~1.DOC) để tương thích ngược.
 - o Đây cũng là cách NTFS hỗ trợ **Hard Links (Liên kết cứng)**: Nhiều tên file trong các thư mục khác nhau có thể cùng trỏ về *một MFT record duy nhất*.
- **\$SECURITY_DESCRIPTOR:** Chứa thông tin bảo mật (Access Control List - ACLs) quy định người dùng (User) hoặc nhóm (Group) nào có quyền gì (Đọc, Ghi, Thực thi...) trên tập tin.
- **\$DATA: Nội dung thực sự của tập tin.** Đây là thuộc tính quan trọng nhất và là nơi thể hiện sự khác biệt lớn của NTFS.
- **\$INDEX_ROOT** và **\$INDEX_ALLOCATION:** Được sử dụng bởi thư mục để lập chỉ mục các tập tin bên trong nó (sẽ giải thích ở Mục 4).
- **\$ATTRIBUTE_LIST:** Nếu một tập tin có quá nhiều thuộc tính (ví dụ, bị phân mảnh quá nhiều, hoặc có quá nhiều Hard Link) khiến 1KB MFT record không chứa đủ, thuộc tính này sẽ được tạo ra. Nó chứa các con trỏ chỉ đến các MFT record "mở rộng" khác, nơi lưu trữ các thuộc tính còn lại.

III. Dữ liệu Nội trú (Resident) và Ngoại trú (Non-Resident)

Đây là cách NTFS xử lý thuộc tính \$DATA (nội dung file) và cũng là một trong những tối ưu thông minh nhất của nó:

Dữ liệu Nội trú (Resident Data)

- **Khi nào:** Khi tập tin có kích thước **rất nhỏ** (thường là dưới 700-800 bytes).
- **Cách thức:** Thay vì cấp phát một cluster riêng ngoài vùng Data Area, NTFS nhét luôn toàn bộ nội dung của tập tin vào bên trong thuộc tính \$DATA, ngay trong 1KB của MFT record.
- **Lợi ích:** Truy xuất cực kỳ nhanh. Khi HĐH đọc MFT record để lấy metadata (như tên, ngày tạo), nó **đã có luôn cả nội dung** của tập tin. Không cần phải di chuyển đầu đọc đến một vị trí khác trên đĩa.

Dữ liệu Ngoại trú (Non-Resident Data)

- **Khi nào:** Khi tập tin lớn hơn, không thể nhét vừa vào MFT record.
- **Cách thức:** Thuộc tính \$DATA trở thành "ngoại trú". Thay vì chứa dữ liệu, nó chứa một danh sách các con trỏ, gọi là **Data Runs (hoặc Extents)**.
- **Data Runs:** Đây là một cách lưu trữ con trỏ hiệu quả hơn FAT. Thay vì trỏ đến từng cluster, một data run sẽ mô tả một chuỗi liên tục, ví dụ: "Bắt đầu từ cluster 500, lấy 120 cluster liên tục" và "Bắt đầu từ cluster 8000, lấy 30 cluster liên tục".

- **Lợi ích:** Rất hiệu quả cho các tập tin lớn và giảm đáng kể việc quản lý chuỗi cluster bị phân mảnh.

IV. Các Metafile Quan trọng

16 MFT record đầu tiên là vị trí cho các metafile quản lý hệ thống.

Tên Metafile	MFT Record	Chức năng
\$MFT	0	Chính là tập tin MFT. Record 0 mô tả chính nó (vị trí các cluster của nó).
\$MFTMirr	1	Bản sao lưu (mirror) của 4 MFT record đầu tiên (quan trọng nhất). Nó được đặt ở chính giữa volume để phòng trường hợp phần đầu đĩa bị hỏng vật lý.
\$LogFile	2	Tập tin Nhật ký (Journaling) . Đây là cốt lõi của tính năng phục hồi (recoverability).
\$Volume	3	Chứa thông tin về volume (nhãn đĩa, phiên bản NTFS).
\$. (Root Dir)	5	Thư mục gốc của volume.
\$Bitmap	6	Một bản đồ bit (bitmap) theo dõi trạng thái của tất cả các cluster trên volume (1 bit cho mỗi cluster: 0 = trống, 1 = đã dùng). Giúp NTFS tìm chỗ trống cực nhanh.
\$Boot	7	Chính là Boot Sector. Lưu dưới dạng tập tin đảm bảo nó có thể được định vị và sửa chữa.
\$Quota	9	Quản lý thông tin hạn ngạch (quota) đĩa cho người dùng.

V. Tổ chức Thư mục:

Trong FAT, thư mục là một danh sách liên kết đơn giản. Để tìm 1 tập tin trong thư mục có 50.000 tập tin, HĐH phải đọc tuần tự (linear search, $O(n)$) từ đầu đến cuối.

Trong NTFS, **thư mục cũng là một tập tin đặc biệt**. Nội dung của nó (thuộc tính \$DATA hoặc \$INDEX_ROOT) là một **cấu trúc B-Tree (cây nhị phân cân bằng)**.

- Danh sách các tập tin con bên trong thư mục được sắp xếp và lập chỉ mục theo cấu trúc cây.
- Khi tìm một tập tin, NTFS thực hiện tìm kiếm nhị phân (binary search, $O(\log n)$).
- **Kết quả:** Ngay cả trong một thư mục chứa 50.000 tập tin, NTFS chỉ mất vài bước nhảy (vài I/O) để tìm thấy MFT record của tập tin đó, thay vì phải đọc 50.000 entry như FAT.

VI. Các tính năng cao cấp khác

Ngoài những cấu trúc trên, thiết kế dựa trên thuộc tính (attribute-based) cho phép NTFS hỗ trợ nhiều tính năng mạnh mẽ:

1. **Bảo mật (Security):** Thuộc tính \$SECURITY_DESCRIPTOR (ACLs) cho phép gán quyền truy cập chi tiết đến từng tập tin/thư mục.
2. **Nén (Compression):** Là một thuộc tính (flag) trên MFT record. Khi được bật, HĐH sẽ tự động nén/giải nén dữ liệu khi ghi/đọc.
3. **Mã hóa (EFS - Encrypting File System):** Cũng là một thuộc tính. Dữ liệu được mã hóa trong suốt. Quan trọng: **Nén và Mã hóa là hai thuộc tính loại trừ lẫn nhau** (không thể dùng cả hai trên cùng một file).
4. **Alternate Data Streams (ADS):** Vì \$DATA là một thuộc tính, một tập tin có thể có *nhiều* thuộc tính \$DATA (ví dụ: file.txt:stream1, file.txt:stream2). Tính năng này cho phép nhúng dữ liệu ẩn vào một tập tin.
5. **Volume Shadow Copy Service (VSS):** Cho phép tạo "ảnh chụp nhanh" (snapshot) của volume tại một thời điểm, cực kỳ quan-trọng cho việc sao lưu dữ liệu đang được sử dụng.

VII. Tình Huống và biện pháp cụ thể

1. Mô tả sai về phân vùng (Partition)

Đây là tình huống mà Bảng Phân vùng (MBR hoặc GPT) bị hỏng, khiến hệ điều hành không nhận diện được các ổ đĩa logic (C:, D:...).

- **Vấn đề:** Mất thông tin về vị trí bắt đầu và kích thước của phân vùng NTFS.
- **Chi tiết cách làm:**
 - o Các công cụ phục hồi phân vùng sẽ quét toàn bộ ổ đĩa (từ đầu đến cuối) để tìm kiếm "dấu hiệu" của một Boot Sector.
 - o Đối với NTFS, dấu hiệu đặc trưng là chuỗi "NTFS" tại offset 3 của một sector.

- Khi tìm thấy một Boot Sector (\$Boot\$), công cụ sẽ đọc các tham số bên trong nó (như kích thước, số sector...) để xác định kích thước của volume.
- Nó cũng có thể tìm kiếm bản sao của MFT (\$MFTMirr\$) để xác nhận tính toàn vẹn.
- Sau khi quét và tìm thấy phân vùng, công cụ sẽ cho phép bạn **ghi lại (rebuild)** Bảng Phân vùng MBR hoặc GPT một cách chính xác.

2. Tham số sai của Volume

Tình huống này xảy ra khi Bảng Phân vùng vẫn đúng, nhưng sector đầu tiên của volume (Boot Sector) bị hỏng. Windows thường báo ổ đĩa là **"RAW"** và đòi format.

- **Vấn đề:** Metafile \$Boot (boot sector) bị hỏng, mất các thông số quan trọng như kích thước cluster, vị trí của MFT.
- **Cách giải quyết**

khôi phục toàn bộ VBR (512 bytes) từ một bản sao lưu.

Offset (Hex)	Kích thước	Tên tham số (BPB)	Tác động nếu thay đổi
0x10003	8 Bytes	OEM ID	Giá trị mặc định là "NTFS ". Nếu thay đổi giá trị của nó, Windows sẽ nhìn vào và nói "Đây không phải NTFS", sau đó báo lỗi "Format disk".
0x1000B	2 Bytes	Bytes Per Sector	Giá trị phải là 00 02 (tức 512 bytes). Nếu đổi (ví dụ 00 04 = 1024), toàn bộ phép toán tìm cluster sẽ sai. HDD sẽ không thể đọc.
0x1000D	1 Byte	Sectors Per Cluster	Đây là một tham số cực kỳ quan trọng. Thường là 08 (8 sector/cluster = 4KB). Nếu bạn đổi nó thành 01 hoặc 04, HDD sẽ tính toán sai địa chỉ của mọi tập tin.
0x10030	8 Bytes	\$MFT Cluster Start	Đây là mục tiêu "chí mạng" nhất. Giá trị này là con trỏ chỉ đến cluster bắt đầu của \$MFT. Nếu bạn đổi nó về 00 00 00 00 00

			00 00 00, HĐH sẽ không biết \$MFT của volume nằm ở đâu.
0x100FE	2 Bytes	Boot Signature	Giá trị phải là 55 AA. Đây là dấu hiệu "sector có thể khởi động". Nếu bạn đổi nó thành 00 00, nhiều hệ thống sẽ từ chối mount (gắn) volume.

May mắn thay, hệ thống file NTFS được thiết kế với khả năng phục hồi. **Nó tự động lưu một bản sao lưu (backup) của VBR ở sector cuối cùng của volume.**

- Đọc **MBR (Master Boot Record)** ở đầu file VHD để tìm ra phân vùng (partition) bắt đầu từ sector nào và có tổng cộng bao nhiêu sector.
- Tính toán vị trí byte của VBR chính (bị hỏng) và VBR sao lưu (còn tốt).
- Đọc và lưu 512 bytes từ vị trí sao lưu.
- Ghi đè 512 bytes đó lên vị trí của VBR chính bị hỏng.

3. Bảng thư mục và bảng Cluster sai

Trong NTFS, khái niệm này tương ứng với lỗi trên **Master File Table (\$MFT\$)** và **\$Bitmap**. Đây là lỗi nghiêm trọng vì \$MFT\$ quản lý mọi file và thư mục.

- **Vấn đề:** Các MFT record bị ghi đè, tham nhũng, hoặc file \$Bitmap (quản lý cluster trống/đã dùng) bị lỗi.
- **Chi tiết cách làm:**
 - o **Sử dụng MFT Mirror:** NTFS lưu một bản sao của 4-16 record MFT đầu tiên trong metafile \$MFTMirr. Nếu lỗi chỉ xảy ra ở các record MFT quan trọng này, ta có thể dùng bản sao để khôi phục.
 - o **Sử dụng \$LogFile:** Metafile \$LogFile ghi lại các thay đổi (giao dịch) của hệ thống. Chạy chkdsk /f (trên Windows) có thể sử dụng file log này để đưa hệ thống về trạng thái nhất quán, nhưng cũng có rủi ro gây mất thêm dữ liệu.
 - o **Quét MFT (MFT Scan):** Các công cụ chuyên nghiệp sẽ bỏ qua \$MFT (nếu nó hỏng) và quét toàn bộ đĩa để tìm các MFT record (chúng có dấu hiệu đặc trưng là "FILE0"). Chúng sẽ xây dựng lại một cây thư mục ảo từ các record tìm thấy.

4. File/thư mục đã xóa

Khi bạn (Shift) delete một file trên NTFS:

- **Cách thức NTFS xóa:**

- 1. Hệ thống **không** xóa MFT record của file đó.
- 2. Nó chỉ đánh dấu 1 bit trong MFT record đó là "không sử dụng" (in-use = 0).
- 3. Các cluster mà file đó chiếm giữ được đánh dấu là "trống" (free) trong \$Bitmap.
- 4. Tên file bị xóa khỏi B-Tree (chỉ mục) của thư mục cha.
- 5. Dữ liệu (nội dung file) thường vẫn còn nguyên trên các cluster cho tới khi hệ thống ghi đè lên các cluster đó bằng dữ liệu mới. MFT record cũng có thể bị tái sử dụng và bị ghi đè.

- **Chi tiết cách làm (Phục hồi - Undelete):**

- 1. Đọc Boot Sector để biết vị trí của MFT record đầu tiên
- 2. Quét vùng MFT: Quét toàn bộ MFT để tìm các địa chỉ của các MFT record và phân tích nếu cờ "in-use = 0" thì nghĩa là file đó đã bị xóa
- 3. Đọc attribute \$FILE_NAME lấy name_len, name_bytes sau đó decode ra tên file
- 4. Đọc attribute \$DATA record chứa runlist (data runs) — chuỗi (LCN, length) chỉ tới cluster vật lý chứa nội dung file.
- 5. Đọc cluster và ghi ra file: Dùng $\text{offset} = \text{LCN} \times \text{bytes_per_cluster}$ để đọc toàn bộ cluster theo runlist và lưu thành file phục hồi.

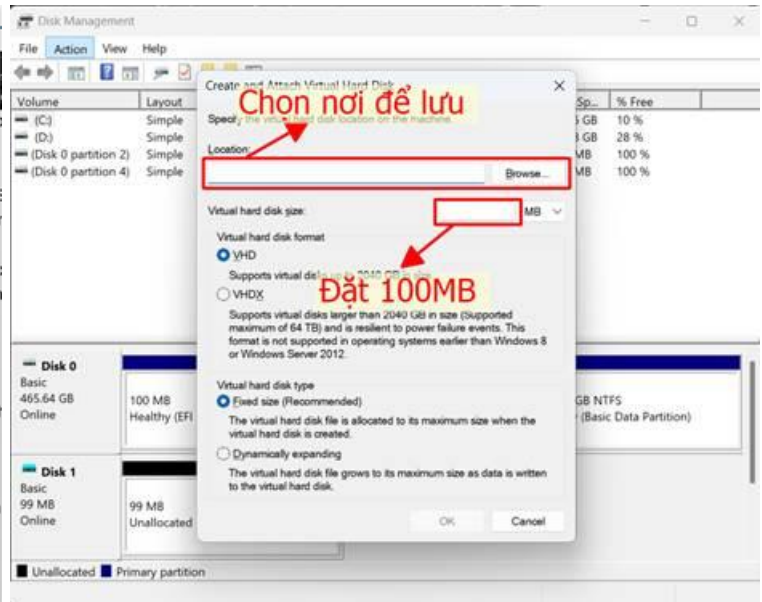
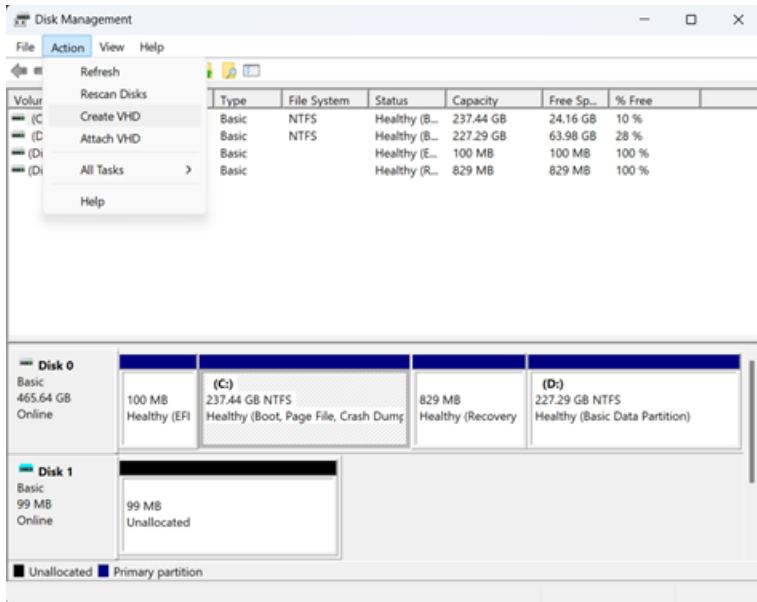
VIII. Demo gây hư hại và phục hồi

1. Mô tả sai về phân vùng (Partition)

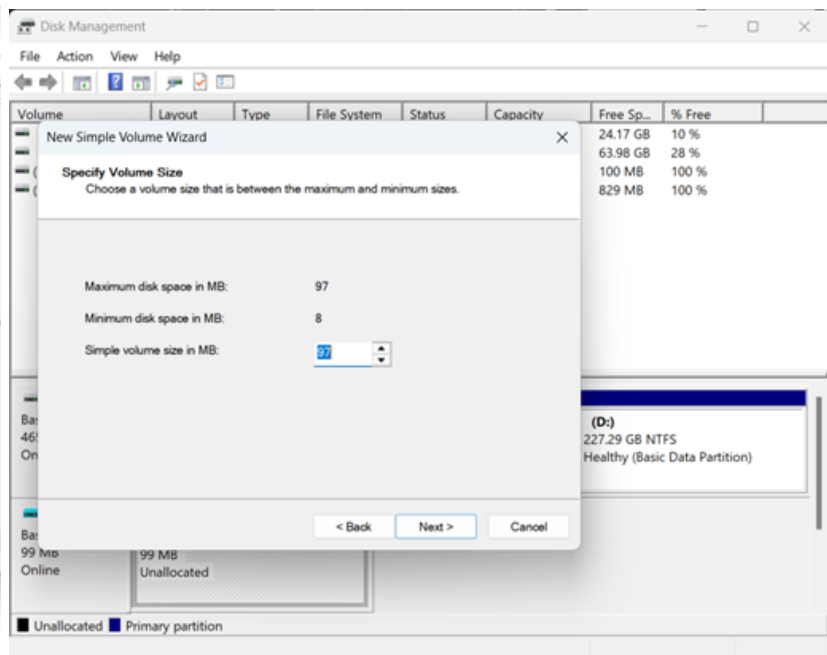
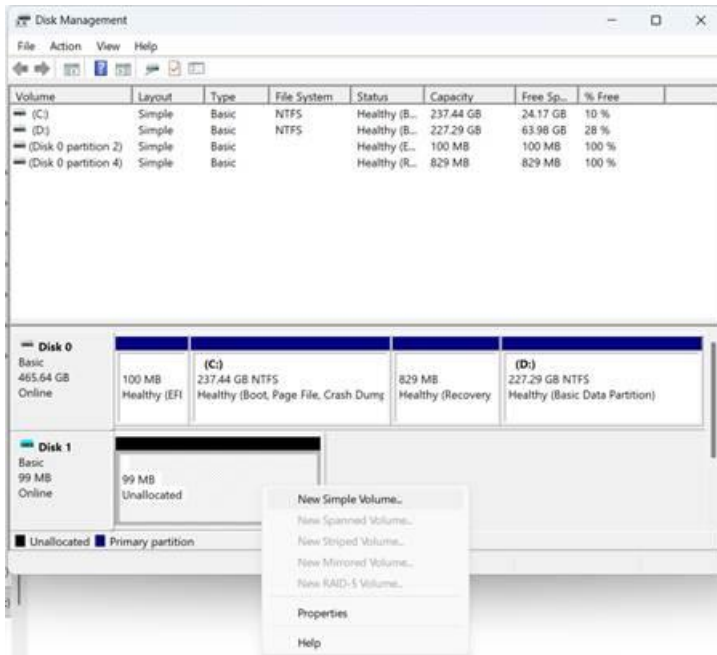
Note: ở lab này em sẽ sử dụng Disk Genius để hỗ trợ thực hiện.

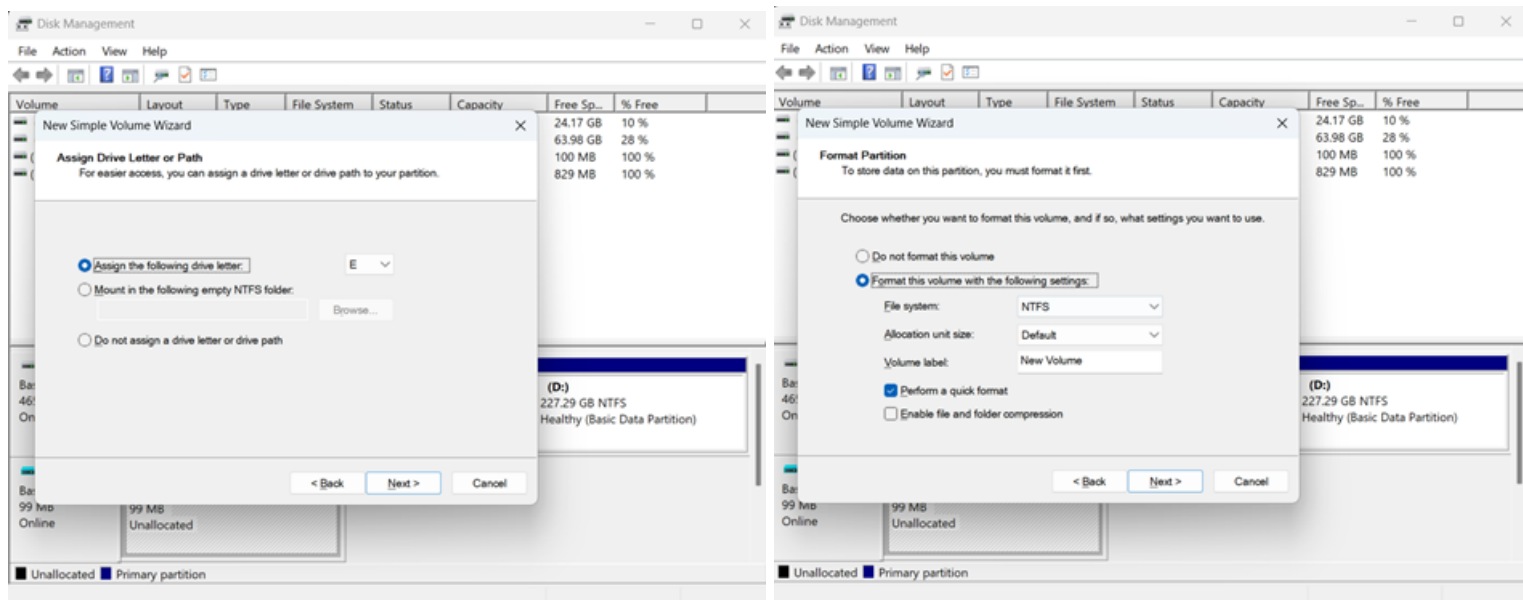
Tạo ổ đĩa ảo:

- Sử dụng Disk Management để tạo một ổ đĩa ảo.

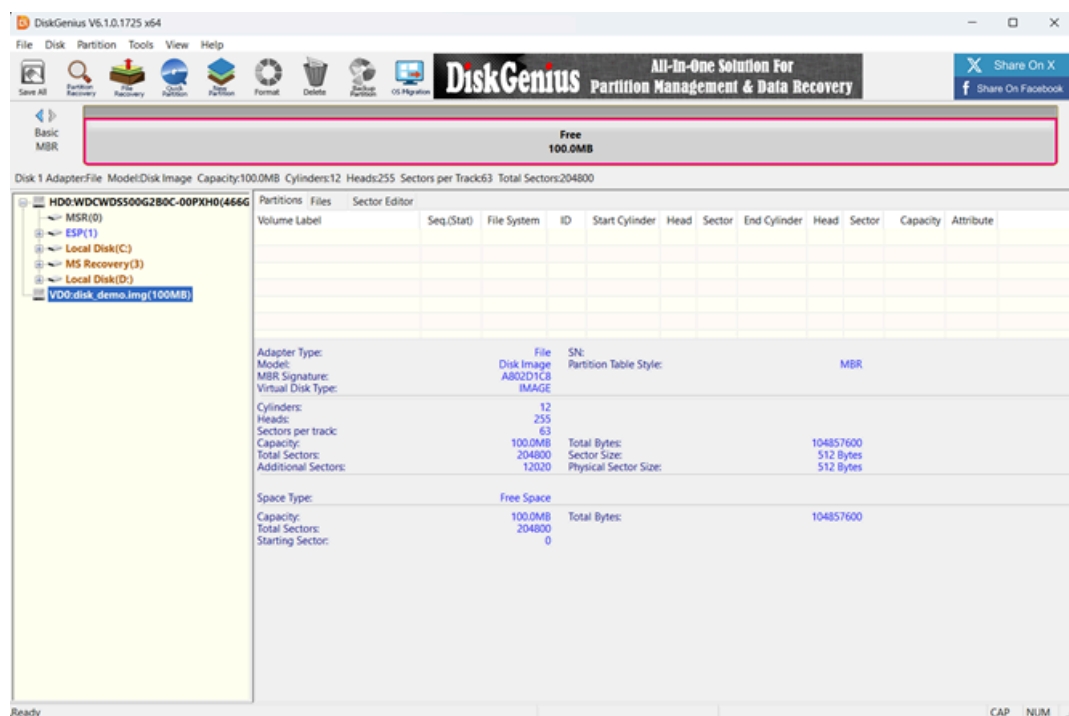


- Chuẩn bị Partition:

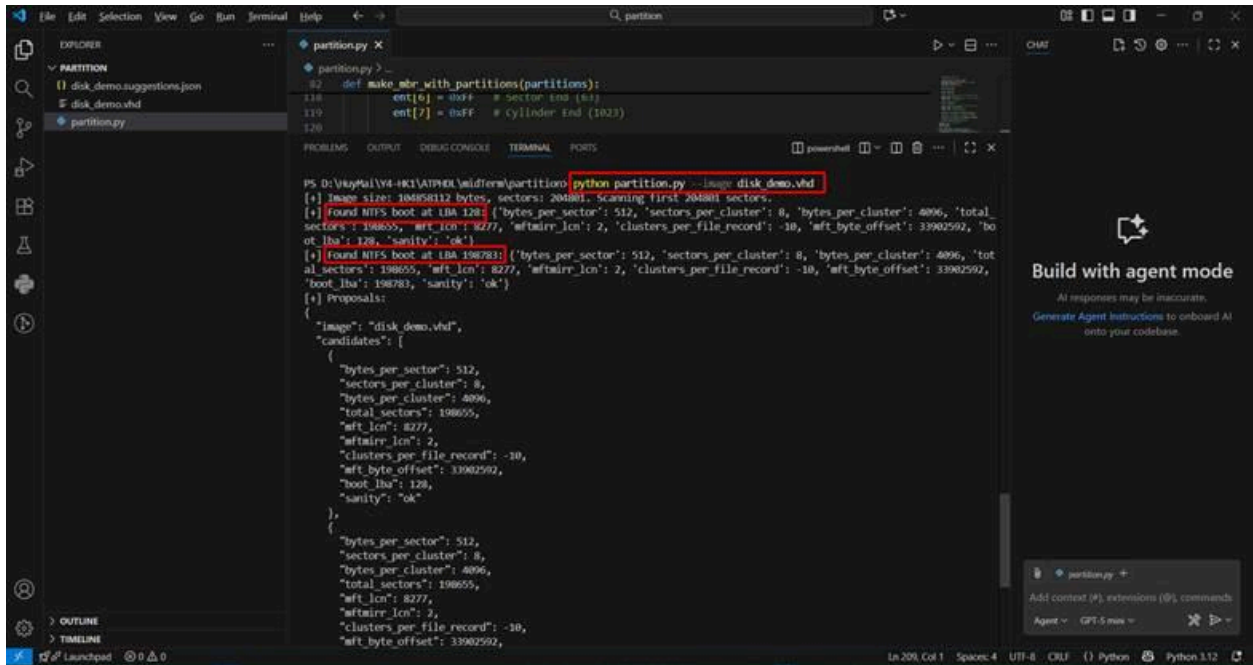




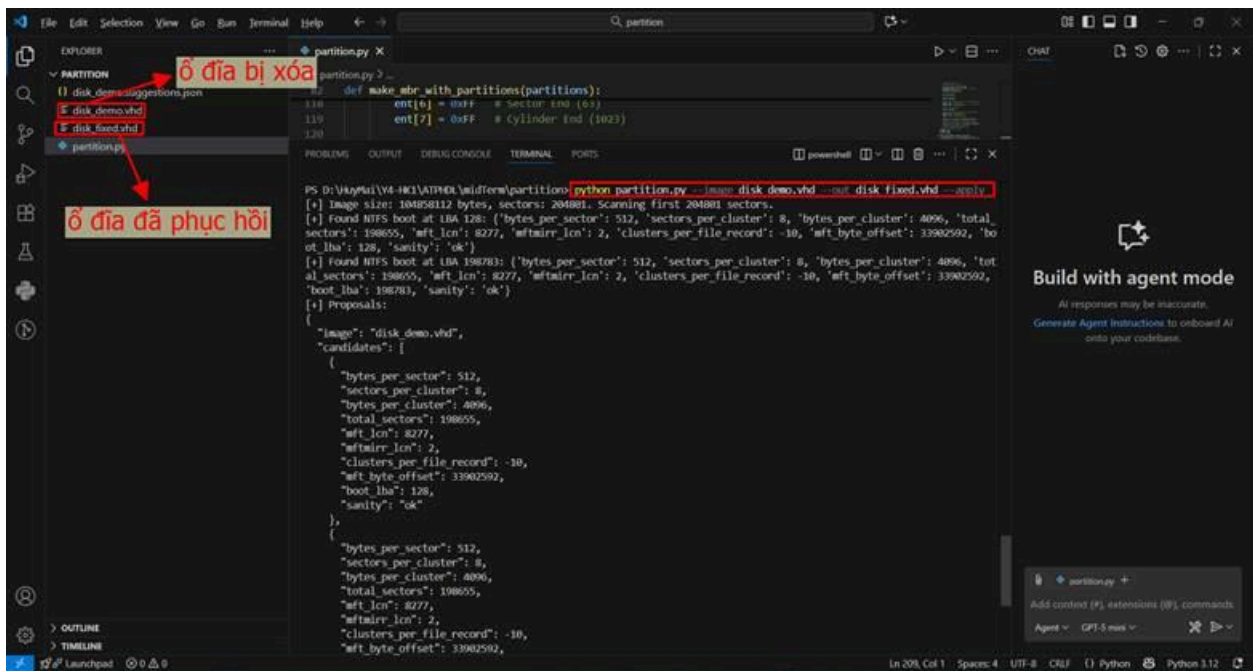
- Ở trong disk genius, chọn disk và xóa partition để mô phỏng tình huống, lúc này phân vùng sẽ biến mất nhưng dữ liệu NTFS vẫn nằm nguyên trong file .img. Hành động này tương đương với việc sử dụng HxD để mở file, sau đó tìm nơi lưu partition và đổi tất cả chúng thành 00.



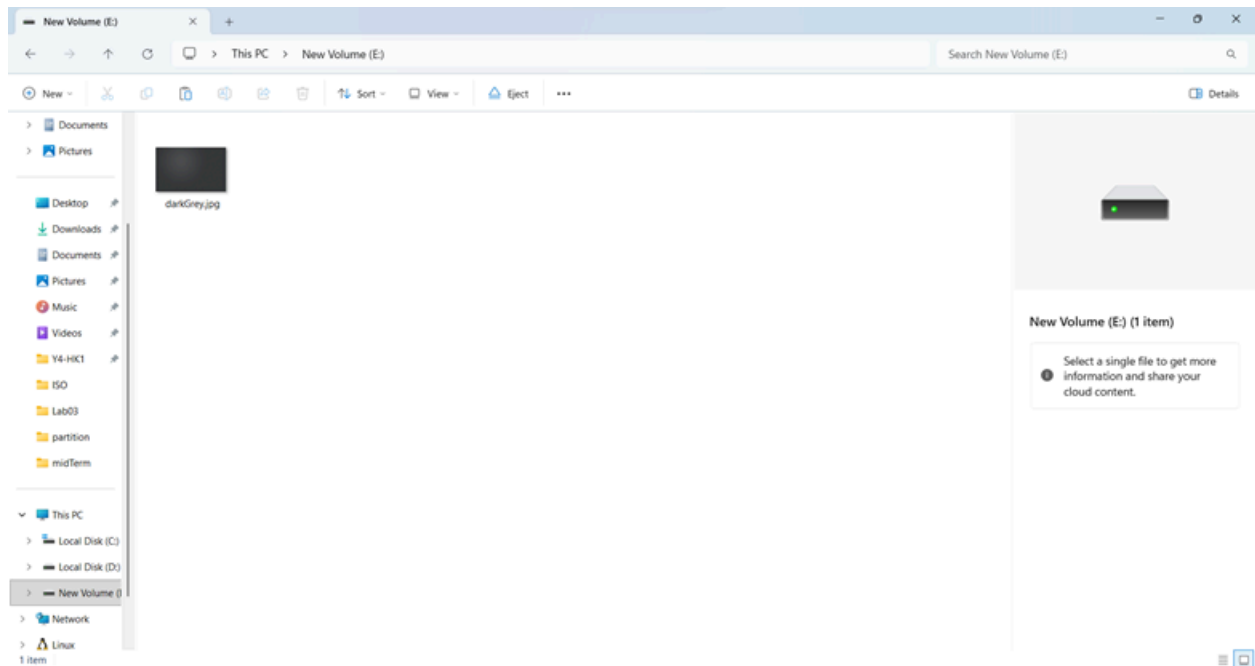
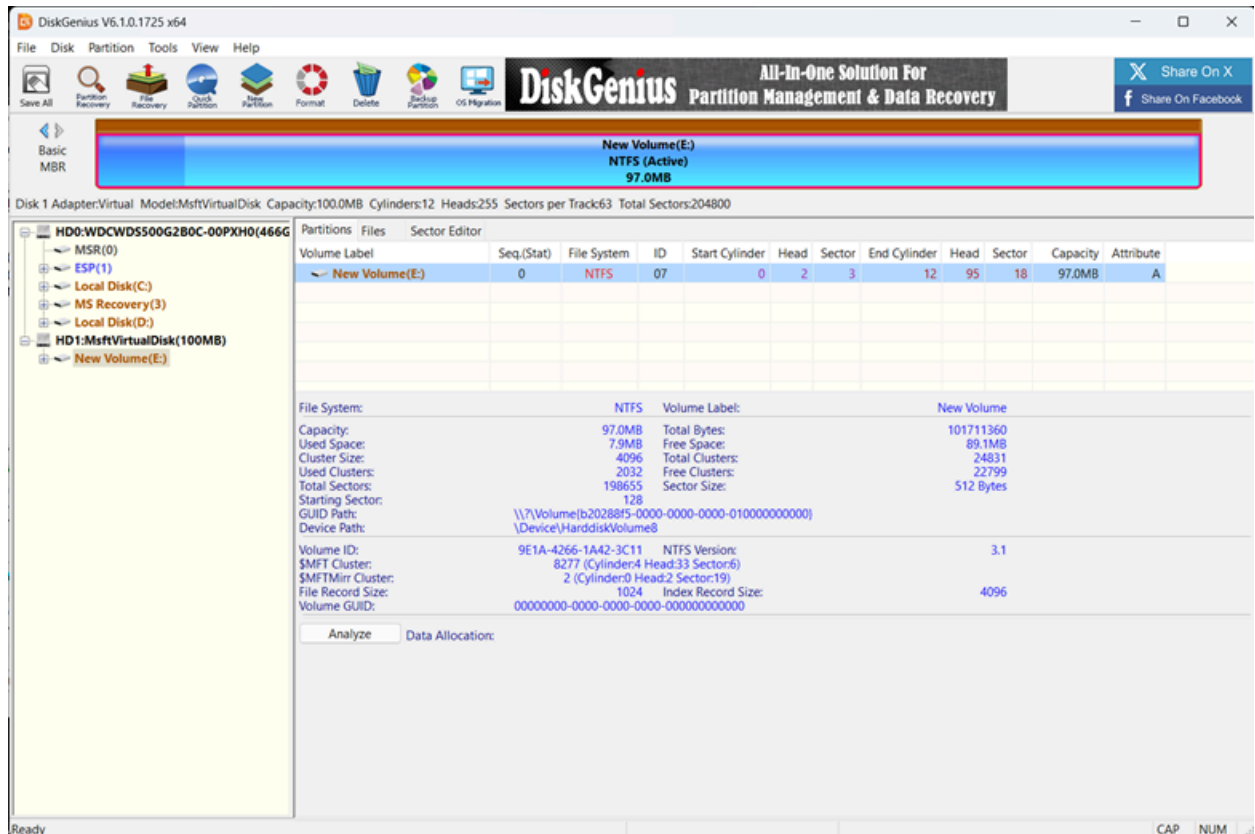
- Copy thư mục disk_demo.vhd vào thư mục chứa file code.
- Chạy code để tìm kiếm thông tin từ file .vhd:



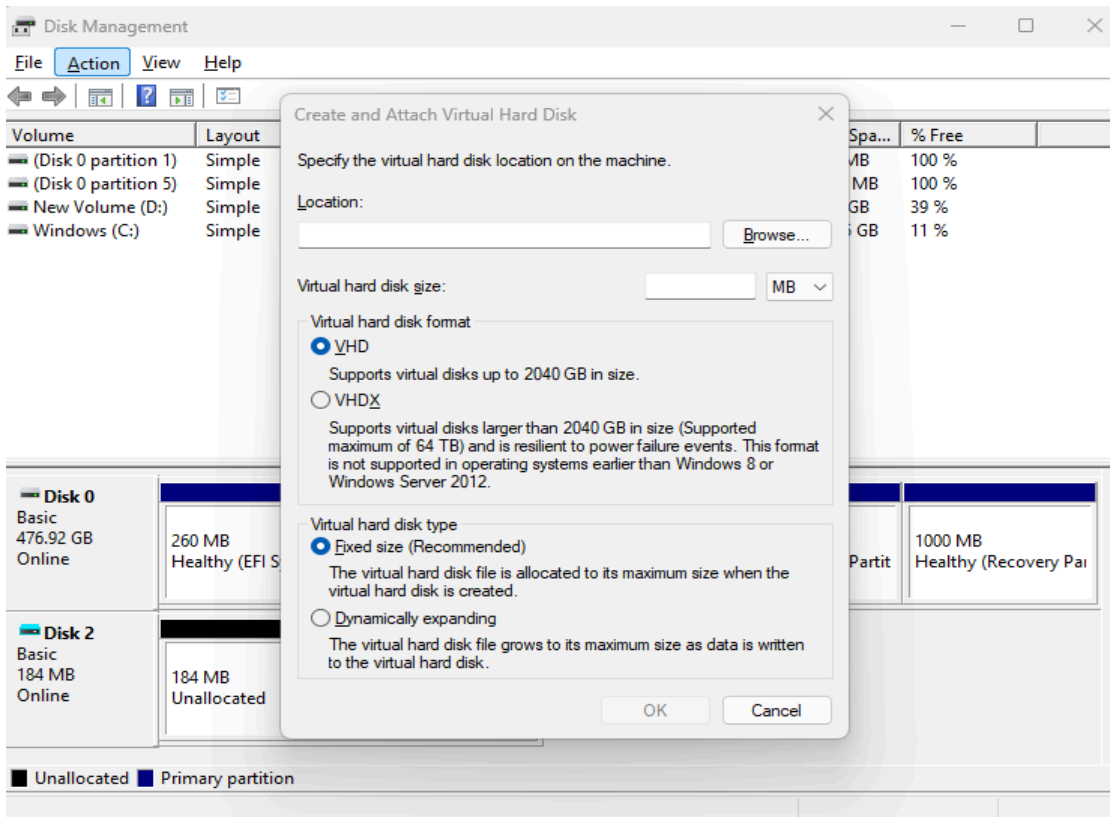
- Tạo ra 1 thư mục .img mới để khôi phục dữ liệu:



- Bây giờ vào lại disk genius để xem thông tin từ ổ đĩa mới phục hồi:

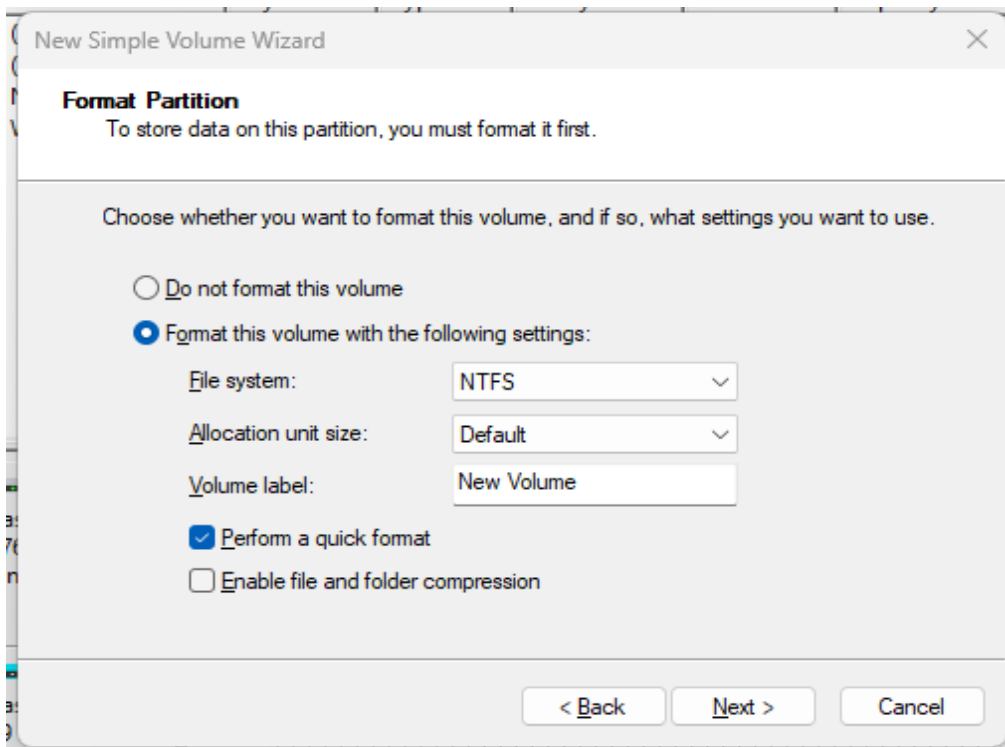


2. Tham số sai của Volume
 Tạo file VHD từ disk management.



Ấn initialize disk → master boot record.

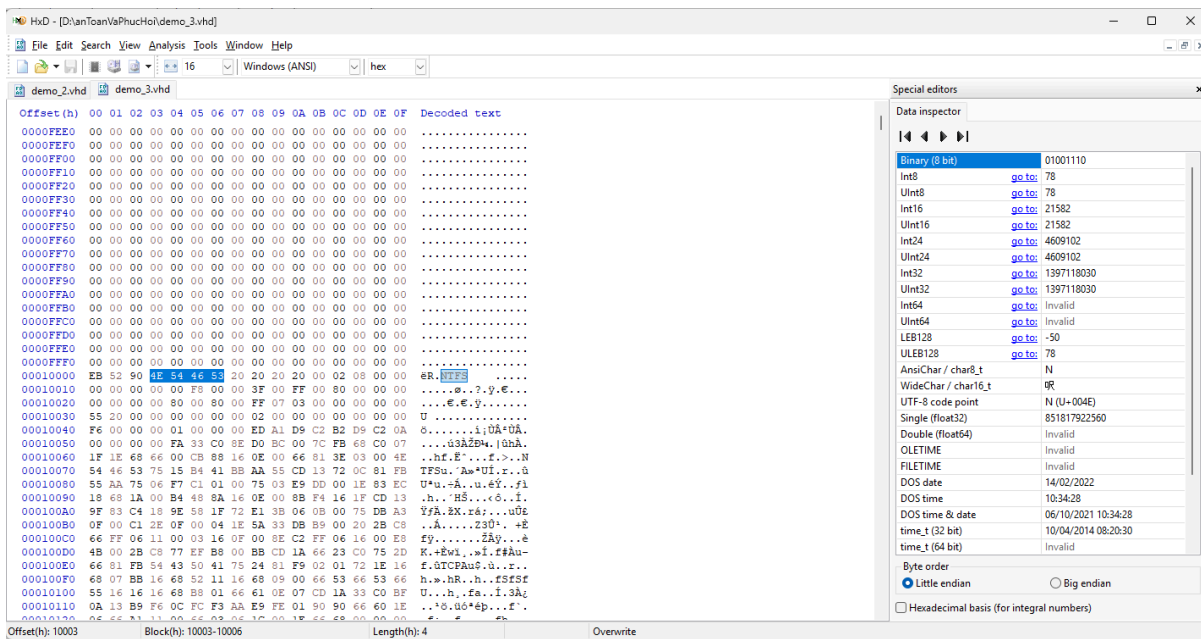
Chọn phân vùng → new simple volume → ở cửa sổ Format Partition chọn NTFS.



Tạo 1 file txt và lưu lại.

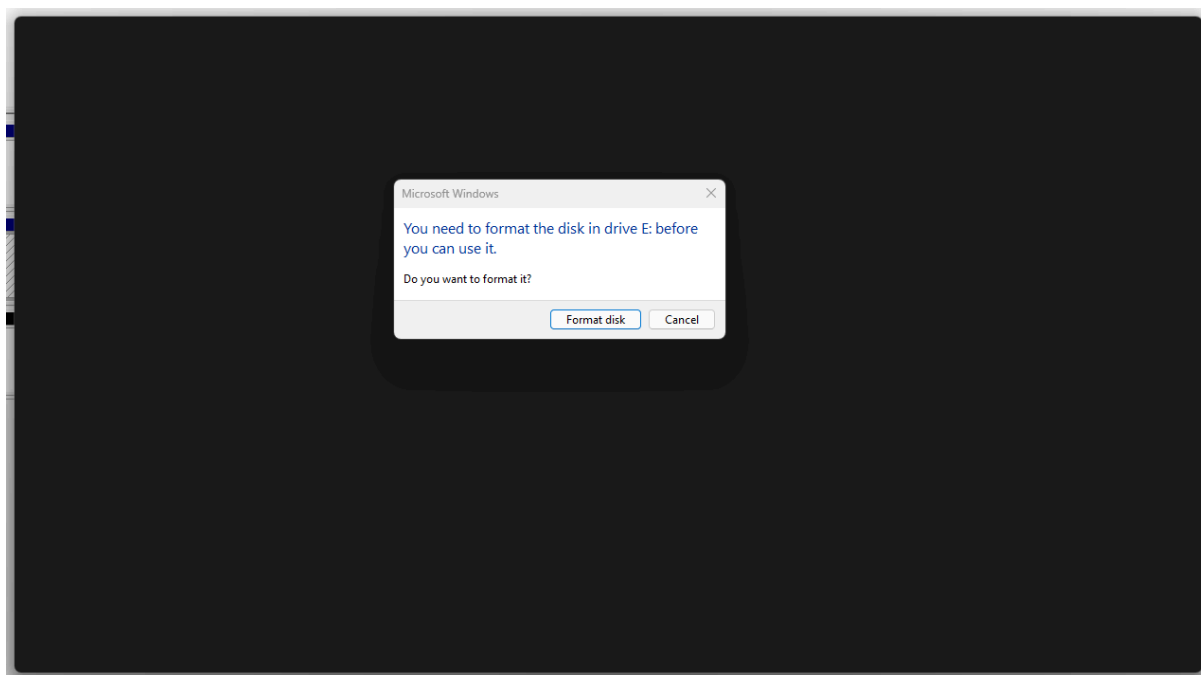
Quay lại disk management và Detach ổ đĩa mới tạo.

Sử dụng HxD mở file demo.vhd và thay đổi một tham số quan trọng của volume NTFS ở offset 10003. Windows sẽ nhìn vào và nói "Đây không phải NTFS" vì thông tin đã bị thay đổi sau đó báo lỗi "Format disk".



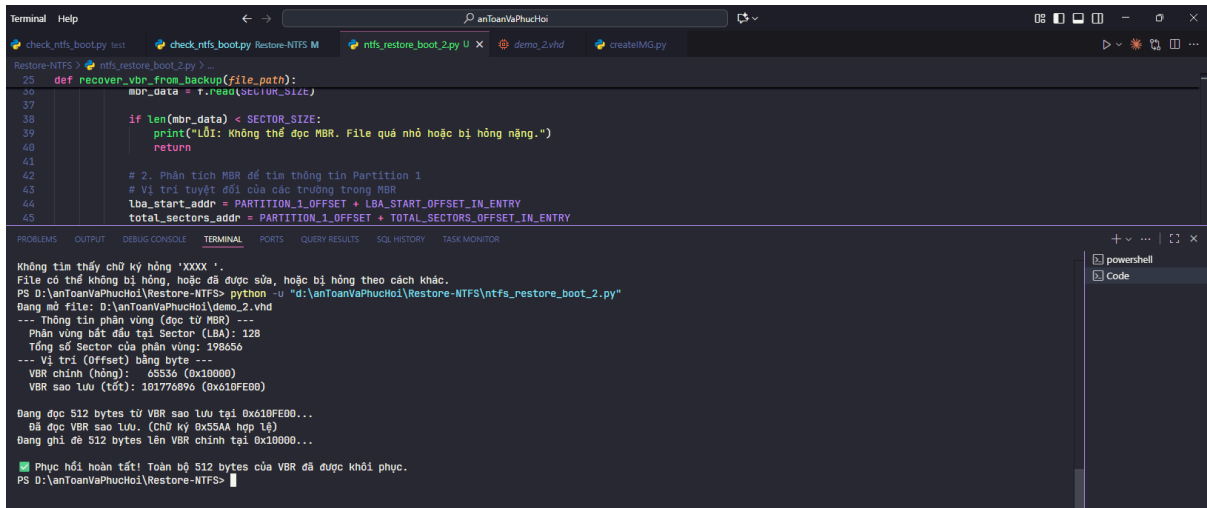
Thay đổi chuỗi 4E 54 46 53 thành 58 58 58 58.

Kết quả:



Quay lại Disk Management và detach demo.vhd

Chạy chương trình dùng để khôi phục toàn bộ phần tham số Volume Boot Record - VBR



```
Terminal Help
check_ntfs_boot.py test check_ntfs_boot.py Restore-NTFS M ntfs_restore_boot_2.py U demo_2.vhd createIMG.py

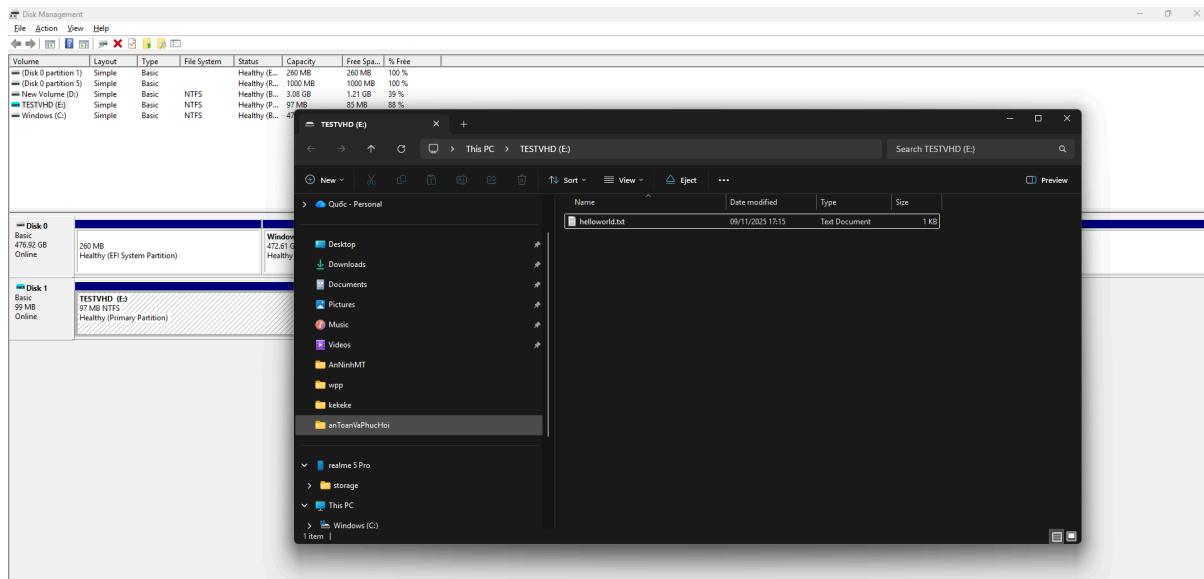
Restore-NTFS > ntfs_restore_boot_2.py > ...
25 def recover_vbr_from_backup(file_path):
30     mbr_data = t.read(SECTOR_SIZE)
37
38     if len(mbr_data) < SECTOR_SIZE:
39         print("Lỗi: Không thể đọc MBR. File quá nhỏ hoặc bị hỏng nặng.")
40         return
41
42     # 2. Phân tích MBR để tìm thông tin Partition 1
43     # Vị trí tuyệt đối của các trường trong MBR
44     lba_start_addr = PARTITION_1_OFFSET + LBA_START_OFFSET_IN_ENTRY
45     total_sectors_addr = PARTITION_1_OFFSET + TOTAL_SECTORS_OFFSET_IN_ENTRY

Không tìm thấy chữ ký hỏng 'XXXX'.
File có thể không bị hỏng, hoặc đã được sửa, hoặc bị hỏng theo cách khác.
PS D:\anToanVaPhuchoi\Restore-NTFS> python -u "d:\anToanVaPhuchoi\Restore-NTFS\ntfs_restore_boot_2.py"
Đang mở file: D:\anToanVaPhuchoi\demo_2.vhd
--- Thông tin phân vùng (đọc từ MBR) ---
Phân vùng bắt đầu tại Sector (LBA): 128
Tổng số Sector của phân vùng: 198656
--- Vị trí (Offset) bằng byte ---
VBR chính (hỏng): 0x5536 (0x10000)
VBR sao lưu (tốt): 101776896 (0x610FE00)

Đang đọc 512 bytes từ VBR sao lưu tại 0x610FE00...
Đã đọc VBR sao lưu. (Chữ ký 0x55AA hợp lệ)
Đang ghi đè 512 bytes lên VBR chính tại 0x10000...

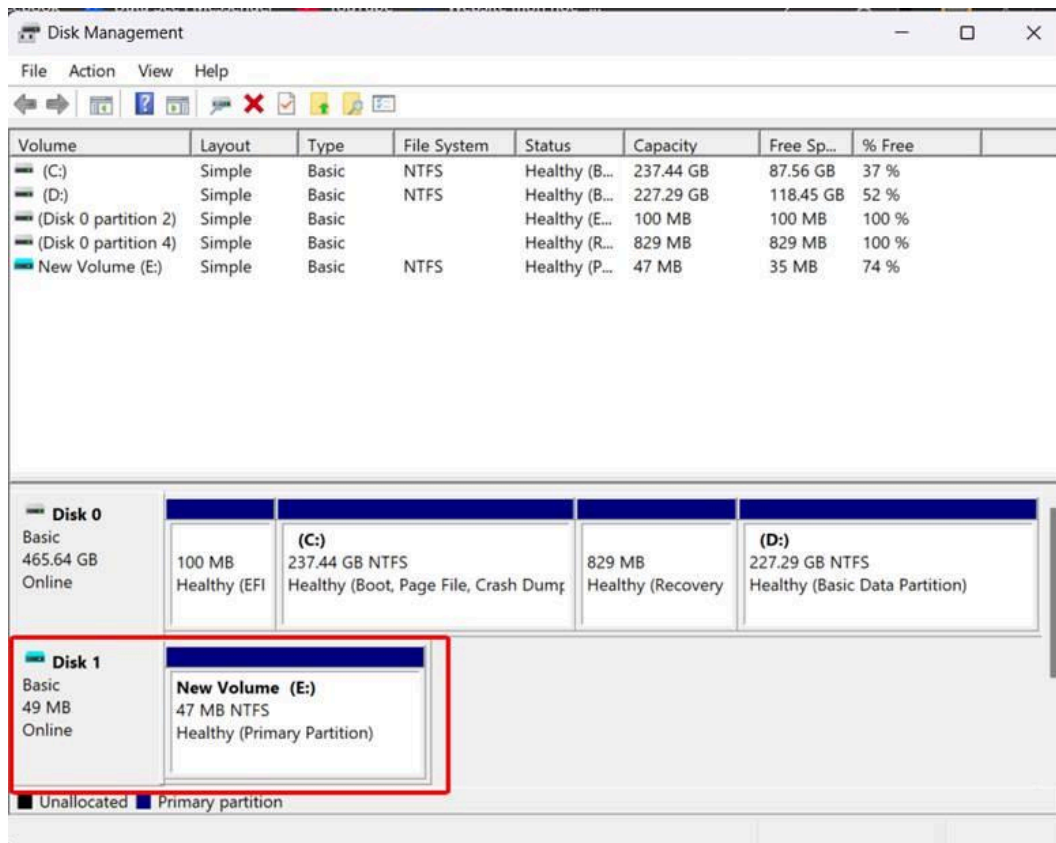
✅ Phục hồi hoàn tất! Toàn bộ 512 bytes của VBR đã được khôi phục.
PS D:\anToanVaPhuchoi\Restore-NTFS>
```

Khôi phục thành công:

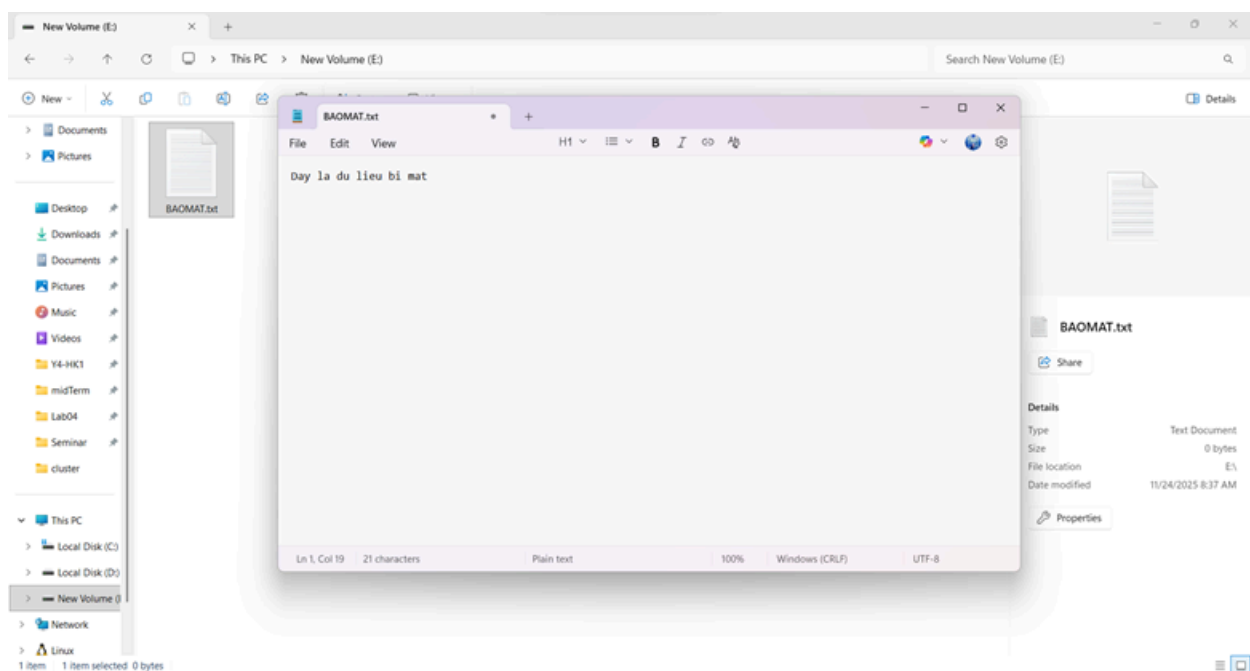


3. Bảng thư mục và bảng Cluster sai

Tạo file .img bằng disk management:

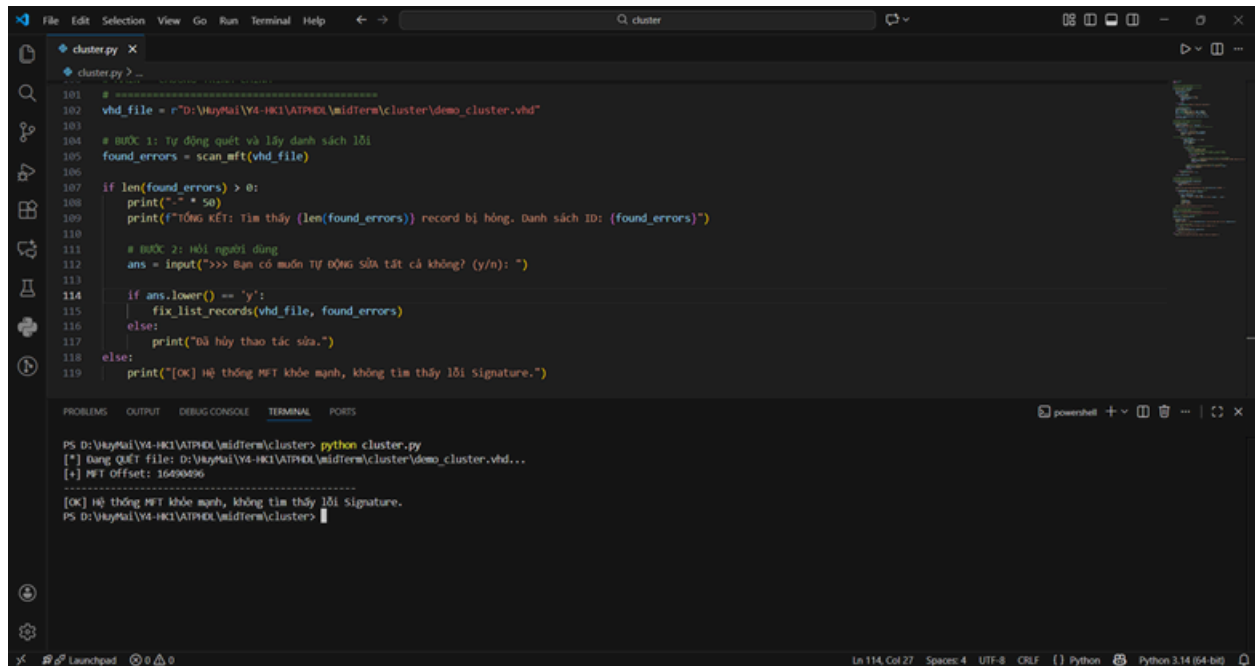


Tạo dữ liệu mẫu: trong ổ đĩa vừa tạo, tạo 1 file BAOMAT.txt chứa nội dung là “Đây là dữ liệu bí mật.”



Detach VHD để tạo lại file .vhd phục vụ cho demo.

Chạy code với file vừa tạo, ta sẽ có kết quả là:



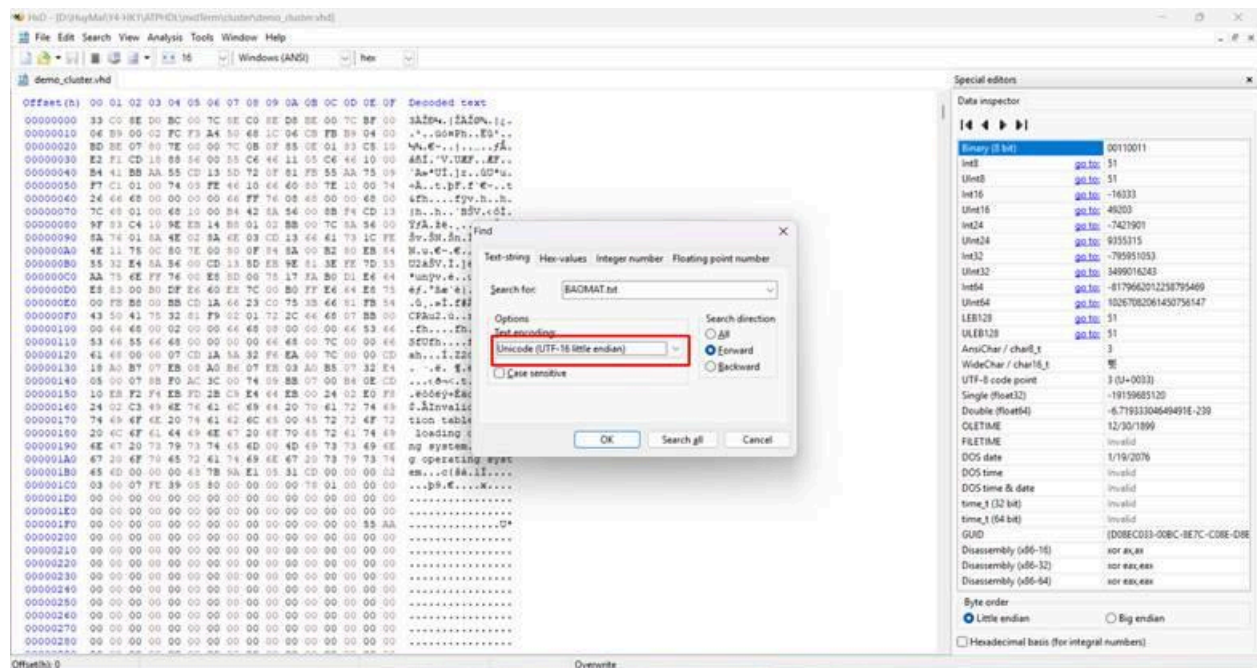
```
101 # .....
102 vhd_file = r"D:\VuyMai\Y4-HK1\ATPHDL\midTerm\cluster\demo_cluster.vhd"
103
104 # BƯỚC 1: Tự động quét và lấy danh sách lỗi
105 found_errors = scan_mft(vhd_file)
106
107 if len(found_errors) > 0:
108     print("-" * 50)
109     print(f"TỔNG KẾT: Tìm thấy {len(found_errors)} record bị hỏng. Danh sách ID: {found_errors}")
110
111     # BƯỚC 2: Hỏi người dùng
112     ans = input(">>> Bạn có muốn TỰ ĐỘNG SỬA tất cả không? (y/n): ")
113
114     if ans.lower() == 'y':
115         fix_list_records(vhd_file, found_errors)
116     else:
117         print("Đã hủy thao tác sửa.")
118 else:
119     print("[OK] Hệ thống MFT khỏe mạnh, không tìm thấy lỗi Signature.
```

```
PS D:\VuyMai\Y4-HK1\ATPHDL\midTerm\cluster> python cluster.py
[*] Đang QUÉT file: D:\VuyMai\Y4-HK1\ATPHDL\midTerm\cluster\demo_cluster.vhd...
[+] MFT Offset: 16490496
.....
[OK] Hệ thống MFT khỏe mạnh, không tìm thấy lỗi Signature.
PS D:\VuyMai\Y4-HK1\ATPHDL\midTerm\cluster>
```

Tool của nhóm em đã đọc được cấu trúc MFT, chứng tỏ bảng quản lý file đang tốt.

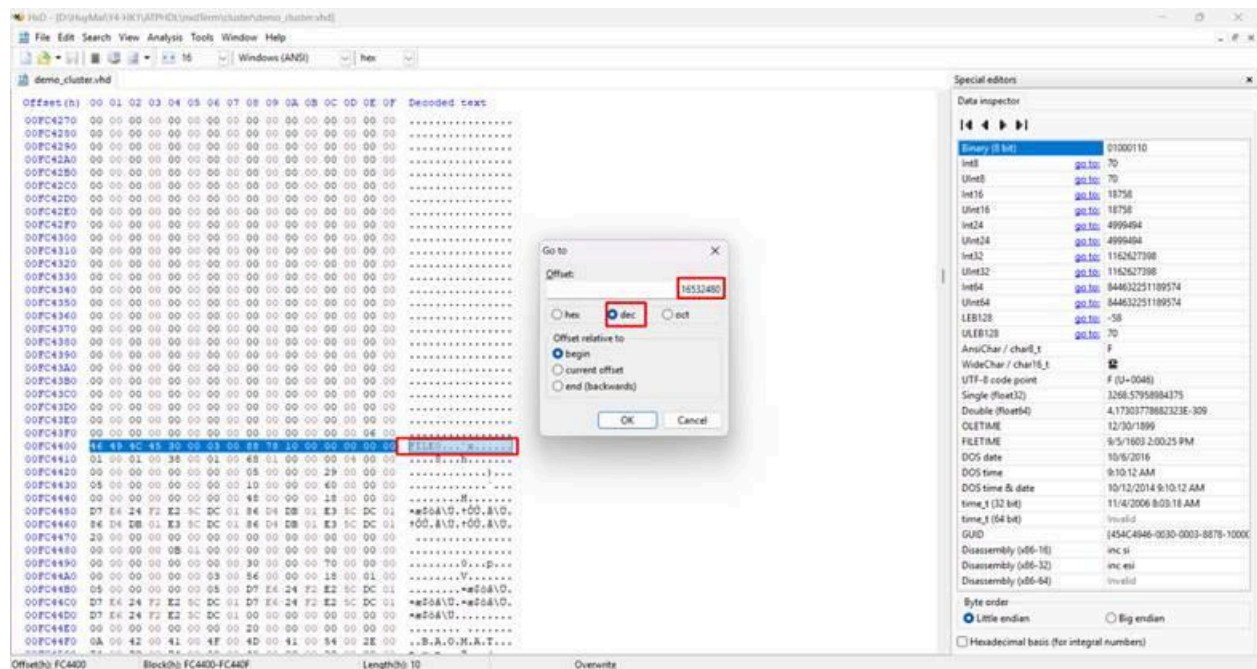
Gây lỗi:

- Sử dụng HxD mở file demo_cluster.vhd lên.
- Tìm file BAOMAT.txt.

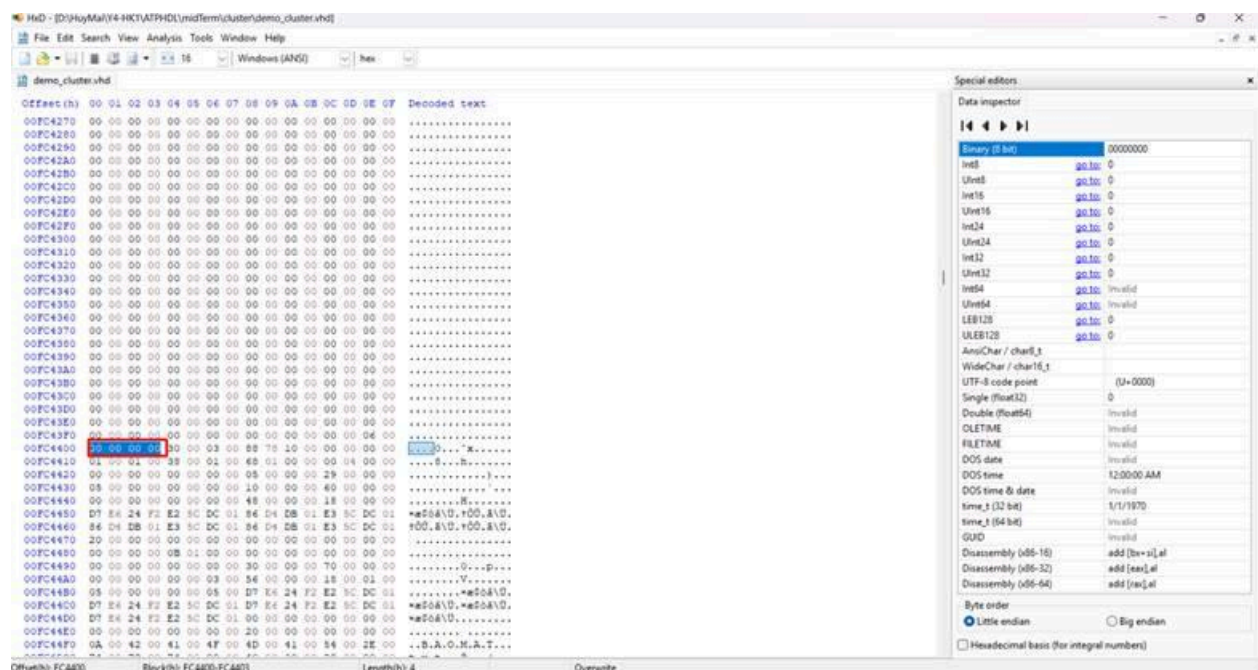


Tìm F.I.L.E bằng cách:

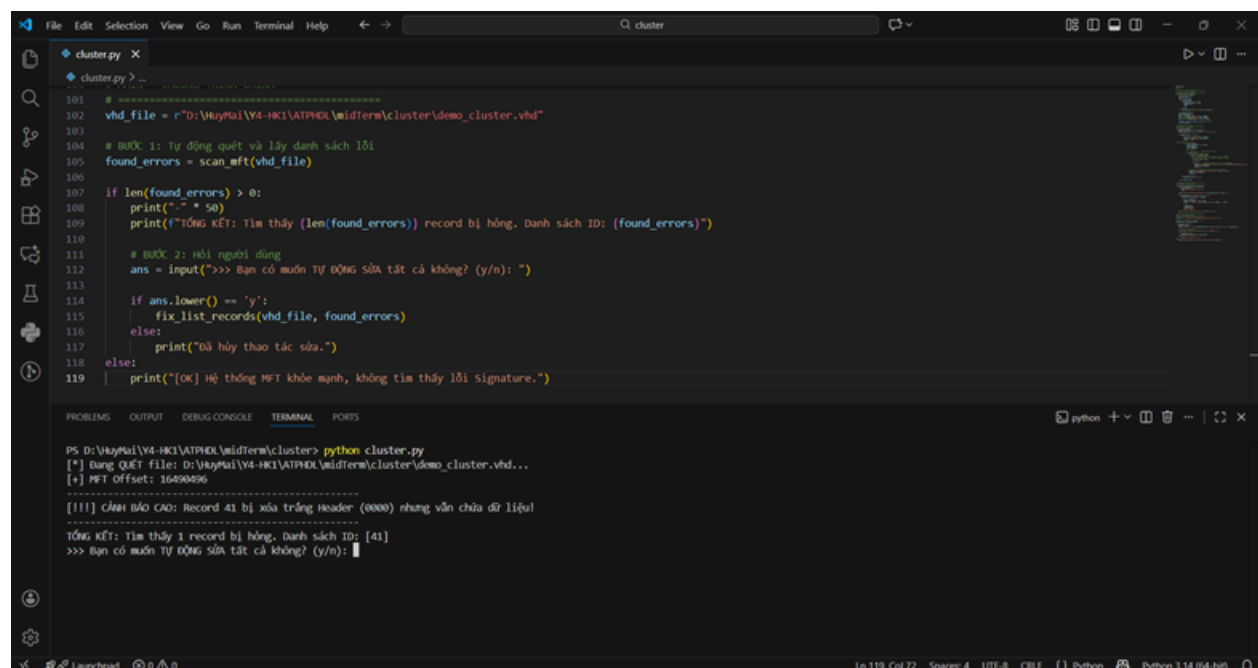
$$- 16490496 + (41 \times 1024) = 16532480$$



Đổi cụm FILE thành 00 00 00 00 (làm hỏng cấu trúc nhận dạng):

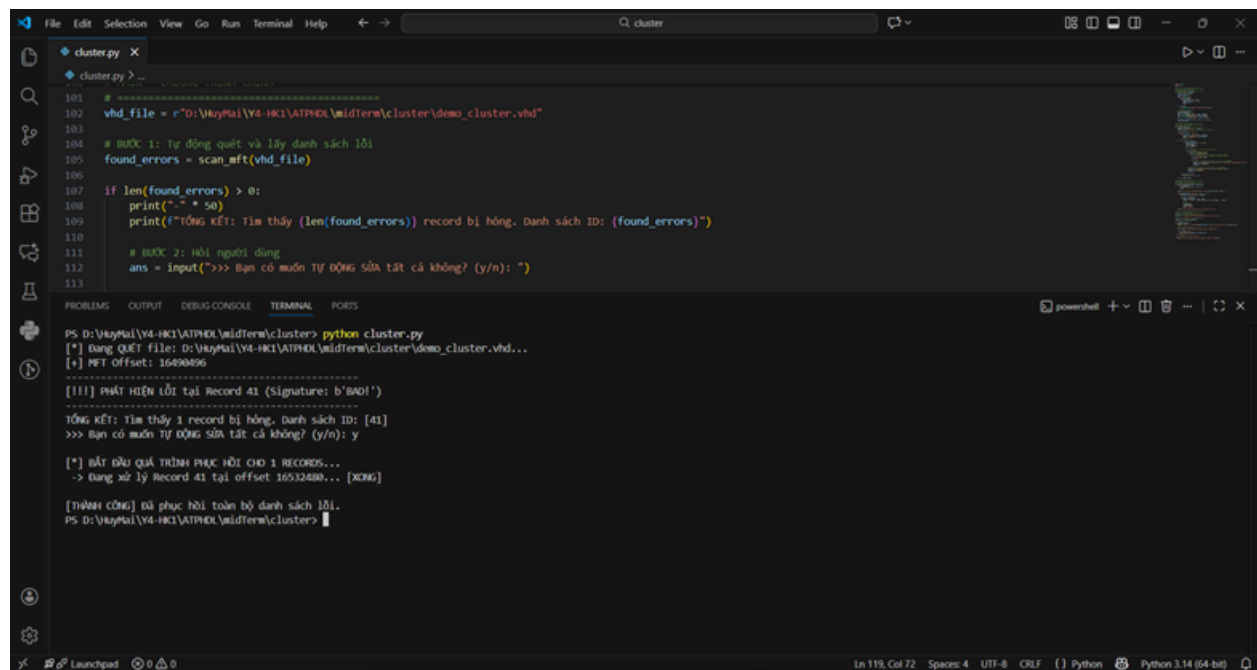
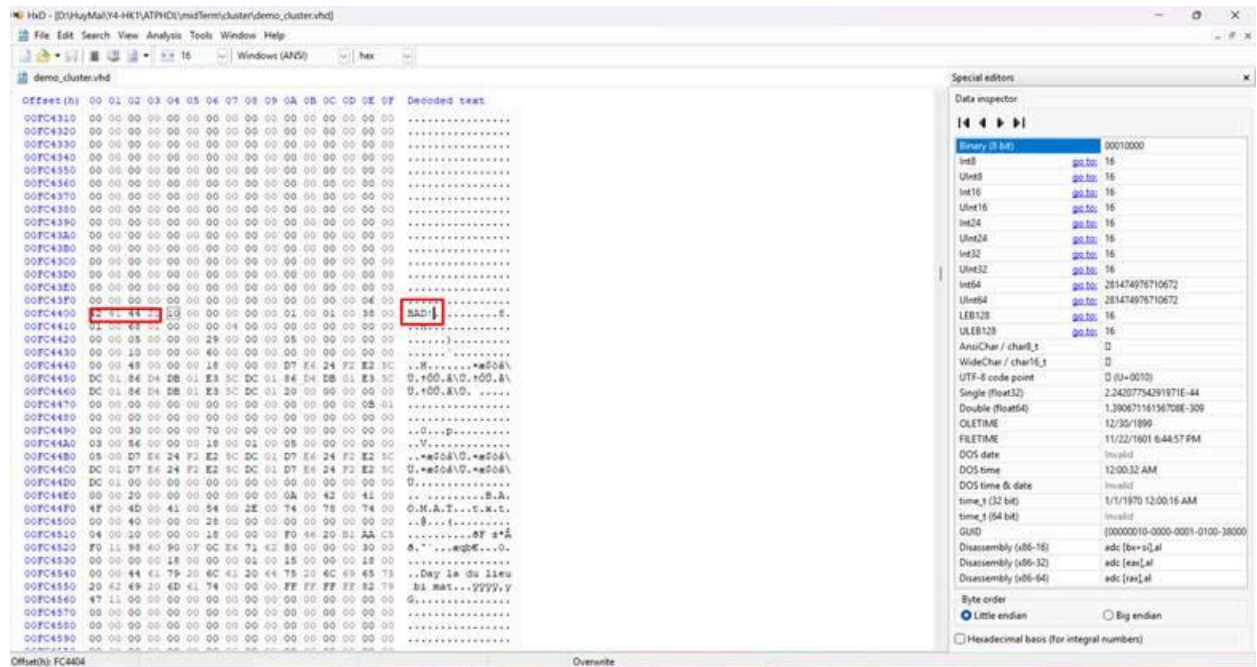


Chạy lại code để tìm lỗi:

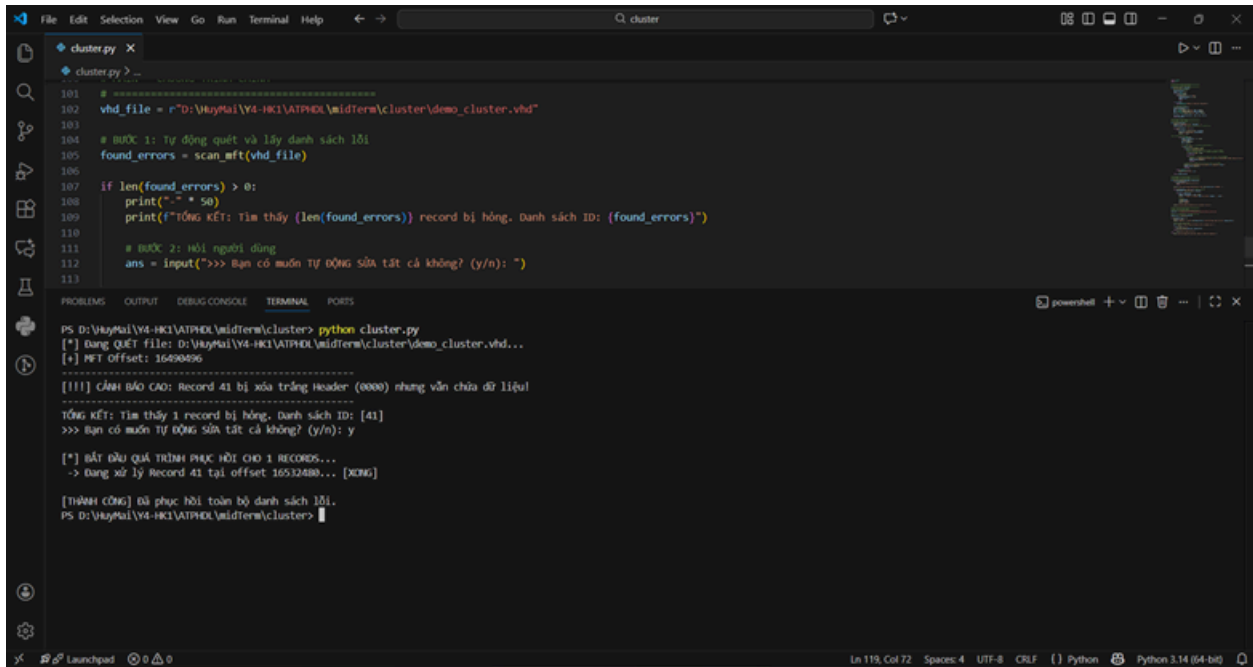


Khi đến Record 41, nó kiểm tra và thấy rằng, đây là một tập trống, nhưng code sẽ kiểm tra tiếp liệu rằng bên trong đó có thật sự trống không, nếu có thì đây là lỗi và nó sẽ phục hồi lại nó.

Ngay cả khi không phải là file trống mà là đổi tên mục:



- Phục hồi:

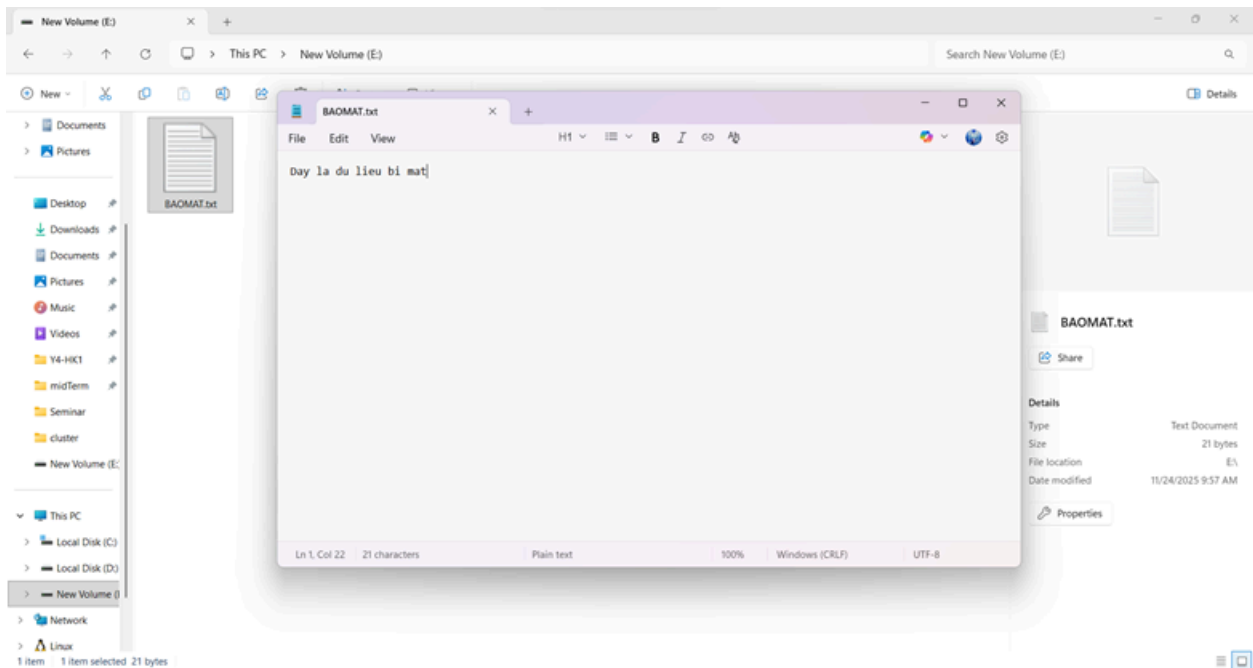


```
cluster.py
101 #
102 vhd_file = r"D:\VuyMai\Y4-HK1\ATPHDL\midTerm\cluster\demo_cluster.vhd"
103
104 # BƯỚC 1: Tự động quét và lấy danh sách lỗi
105 found_errors = scan_mft(vhd_file)
106
107 if len(found_errors) > 0:
108     print("-" * 50)
109     print(f"TỔNG KẾT: Tìm thấy {len(found_errors)} record bị hỏng. Danh sách ID: {found_errors}")
110
111     # BƯỚC 2: Hỏi người dùng
112     ans = input(">>> Bạn có muốn tự động sửa tất cả không? (y/n): ")
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

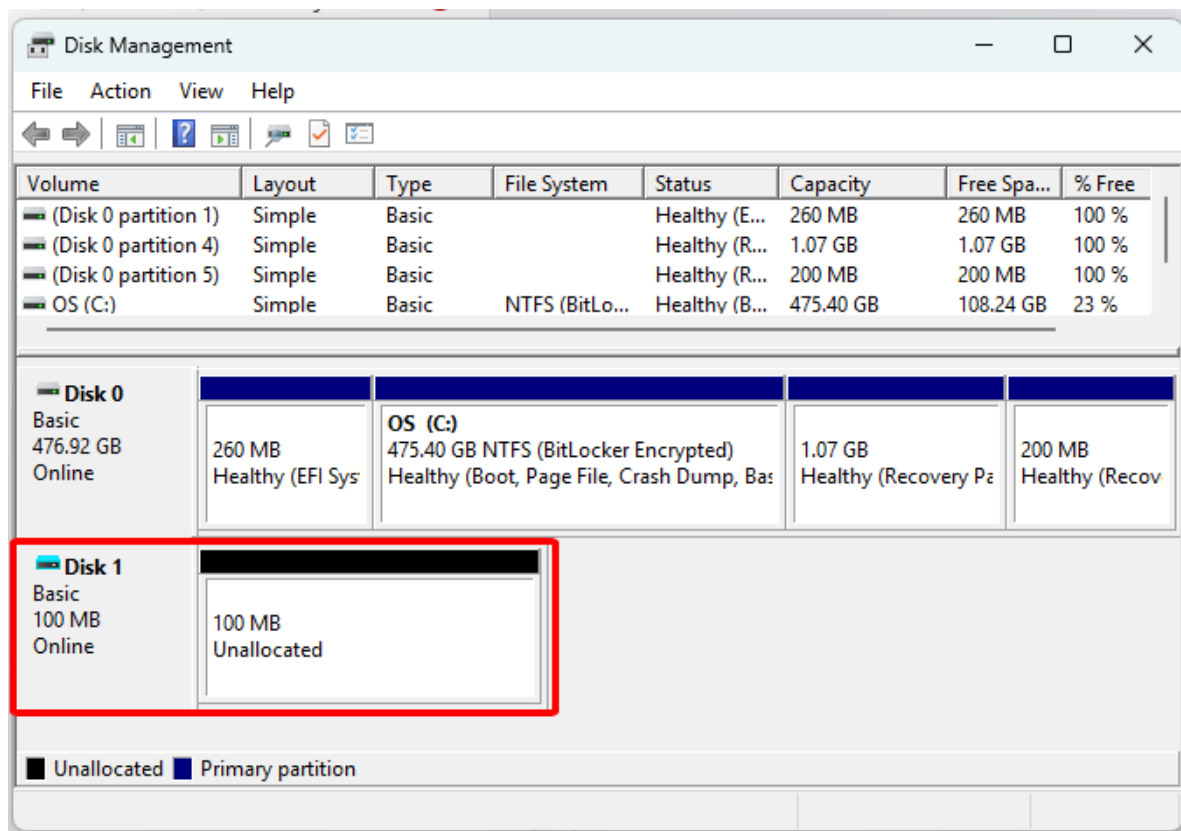
```
PS D:\VuyMai\Y4-HK1\ATPHDL\midTerm\cluster> python cluster.py
[*] Đang QUÉT file: D:\VuyMai\Y4-HK1\ATPHDL\midTerm\cluster\demo_cluster.vhd...
[*] MFT Offset: 16490496
[111] Chạy báo cáo: Record 41 bị xóa trắng Header (0000) nhưng vẫn chứa dữ liệu
TỔNG KẾT: Tìm thấy 1 record bị hỏng, Danh sách ID: [41]
>>> Bạn có muốn tự động sửa tất cả không? (y/n): y

[*] BẮT ĐẦU QUÁ TRÌNH PHỤC HỒI CHO 1 RECORDS...
-> Đang xử lý Record 41 tại offset 16532480... [XONG]

[THÀNH CÔNG] Đã phục hồi toàn bộ danh sách lỗi.
PS D:\VuyMai\Y4-HK1\ATPHDL\midTerm\cluster>
```

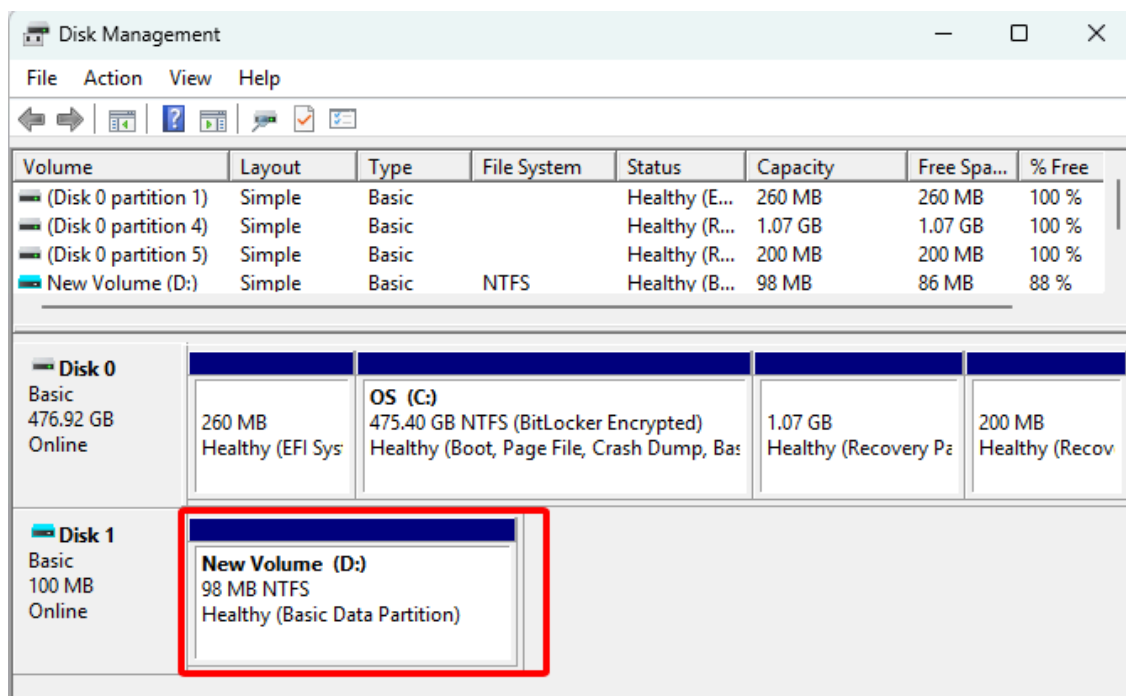


4. File/thư mục đã xóa
Tạo một file .img bằng Disk Management

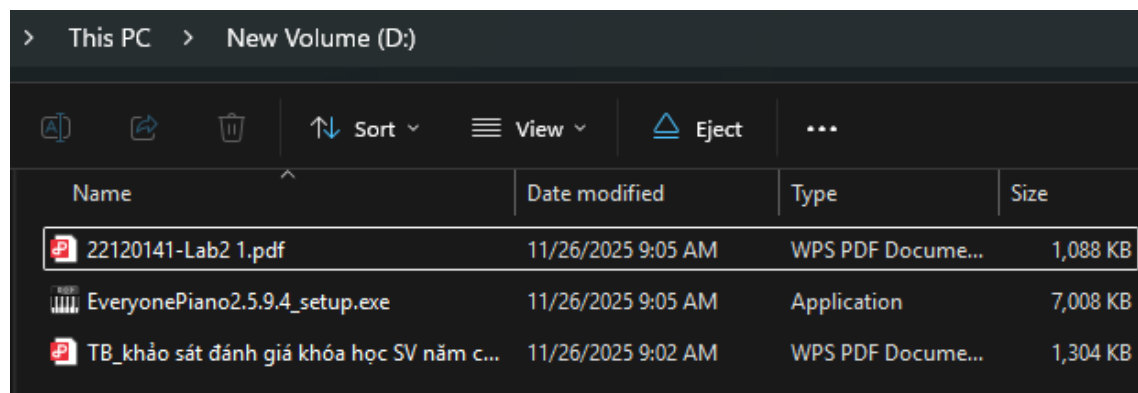


Ấn initialize disk → master boot record

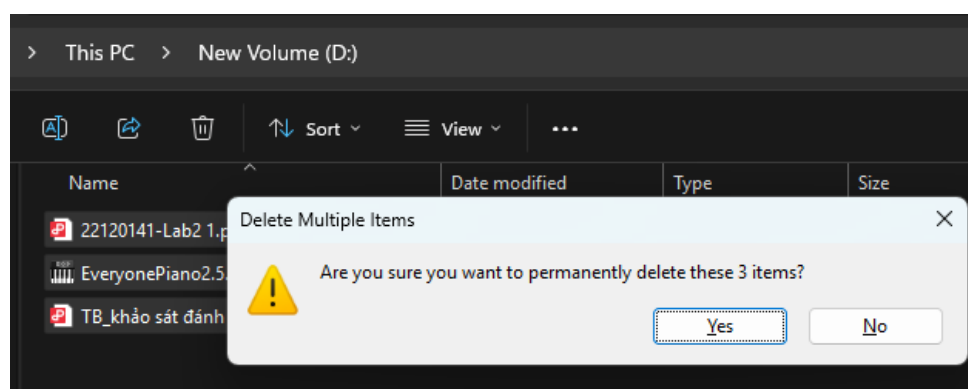
Chọn phân vùng → new simple volume → ở cửa sổ Format Partition chọn NTFS



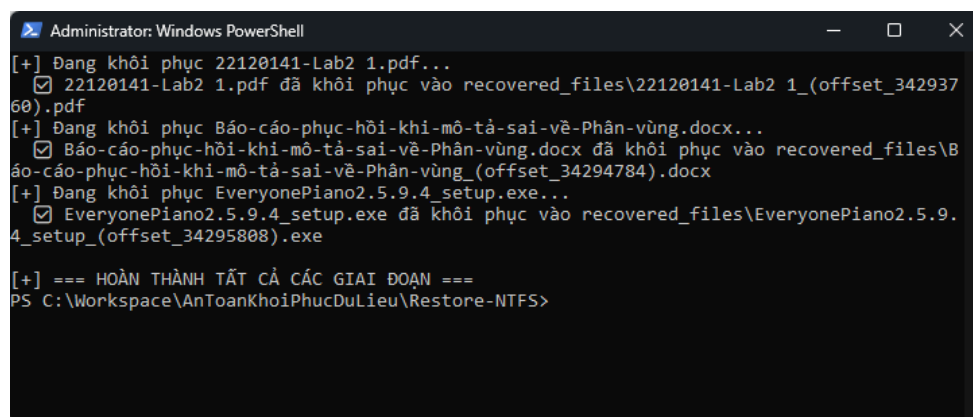
Tiến hành copy 2 file pdf, 1 file exe vào ổ D mới tạo



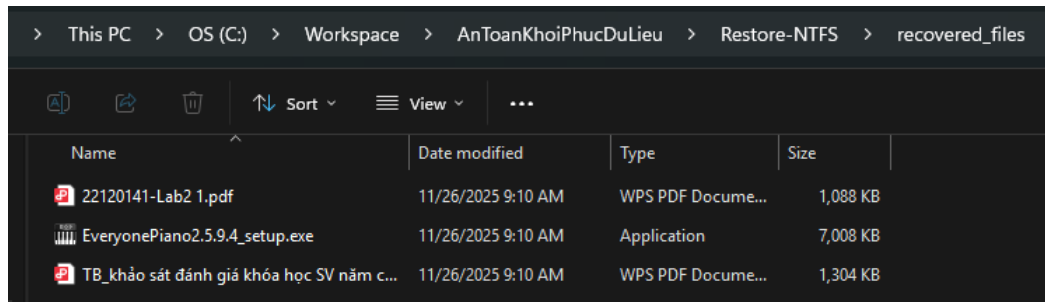
Xóa vĩnh viễn các file trên bằng shift delete



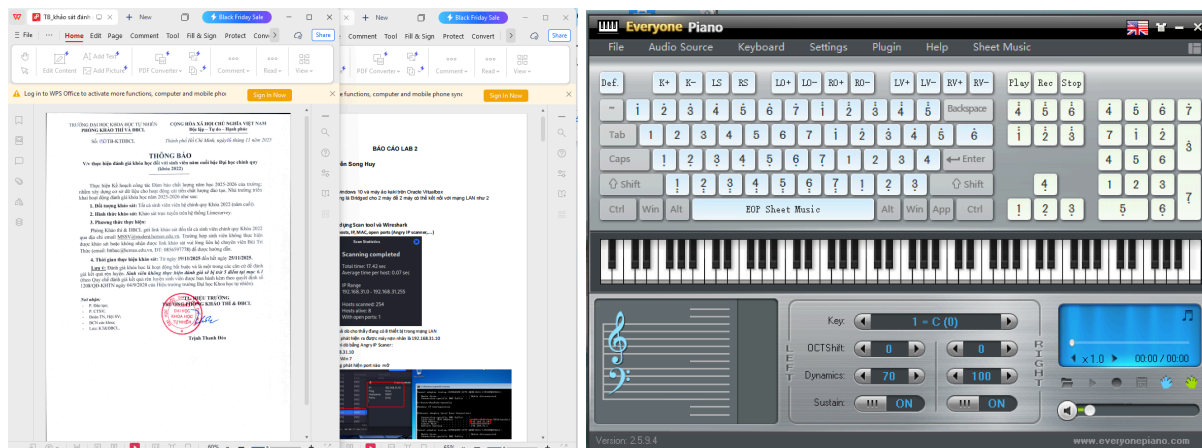
Chạy chương trình python khôi phục bằng Windows PowerShell với quyền admin



Các file tìm được sẽ không được khôi phục tại đó mà ghi lại vào trong một folder nằm cạnh source code



Thử mở 2 file pdf và file exe kiểm tra. Kết quả 3 file đều mở thành công



Kết quả toàn bộ quá trình đã thực hiện đúng và chương trình khôi phục dữ liệu thành công

Tài liệu tham khảo

[AT&PHDL.C3.HeThongTapTin.pdf](#)

[AT&PHDL_C3_Hư hỏng logic trên file system.pdf](#)

Chat GPT, Gemini AI.